

Canadian FHIR Interoperability Engineering Appendix

Deliverables, Repository Structure, Example Code, Test Assets, and Build Blueprint

Companion to the SDLC Addendum and Project Companion Guide

Purpose. This appendix provides a concrete starting package for an engineering prototype. It contains a delivery backlog, repository layout, example Java/Spring Boot/HAPI FHIR structures, example FHIR payloads, infrastructure files, and testing assets. It is intentionally a synthetic-data, MVP-oriented blueprint—not a production deployment or legal compliance assertion.

1. Deliverables Register

ID	Deliverable	Owner	Acceptance evidence
DEL-001	Project charter and MVP scope	Product owner	Approved use case, boundaries, stakeholders, success measures
DEL-002	Standards baseline	FHIR architect	Pinned FHIR R4, PS-CA, CA Core+, CACDI, terminology versions and package references
DEL-003	Clinical workflow and safety scenarios	Clinical informatician	Emergency, medication, lab, identity, partial-response, and downtime flows reviewed
DEL-004	Privacy and threat assessment package	Privacy/security leads	Data flow, risk register, mitigation decisions, no-real-PHI development policy
DEL-005	FHIR implementation guide / profile map	FHIR architect	Supported profiles/resources/searches/operations and validation evidence
DEL-006	CACDI-to-FHIR mapping matrix	FHIR + terminology leads	Each in-scope element mapped to resource/path/value set/source/test
DEL-007	Terminology strategy and service design	Terminology lead	Validation, mapping, versions, original-code preservation, unmapped-code workflow
DEL-008	Identity and duplicate-resolution design	Identity lead	Multiple identifiers, match levels, thresholds, review queue, merge/unmerge audit
DEL-009	Consent and authorization design	Security/privacy leads	SMART, purpose, organization, patient context, consent, break-glass policy
DEL-010	Federated gateway adapter contract	Interoperability architect	Capability discovery, request/response, error, auth, timeout, source status contract
DEL-011	Target architecture and ADRs	Architecture lead	Architecture diagram plus recorded technical decisions
DEL-012	FHIR server MVP	Backend engineer	Synthetic resources, search, \$summary, validation, OperationOutcome responses
DEL-013	Provenance and audit subsystem	Backend/security leads	Provenance captured; audit events complete and queryable
DEL-014	Synthetic data package	QA/clinical informatics	At least ten safety-focused test patients and invalid-resource fixtures
DEL-015	Automated test suite	QA engineer	Unit, integration, profile, terminology, identity, policy, contract, resilience tests
DEL-016	CI/CD and infrastructure	DevOps/SRE	Build/security gates, environment separation, deploy/rollback automation
DEL-017	Observability and incident runbooks	SRE/security	Health, latency, source status, alerts, privacy/security/clinical incident procedures
DEL-018	Pilot readiness package	Product/clinical leads	Training, go-live checklist, support model, metrics dashboard, post-pilot plan

2. Repository and File Structure

Use a monorepo for a student, prototype, or initial pilot. Keep standards, governance, test fixtures, infrastructure, and implementation code versioned together. Do not place PHI, real credentials, production tokens, or unencrypted exports in the repository.

```
canadian-fhir-interoperability/
■■■ README.md
■■■ LICENSE
■■■ .gitignore
■■■ .env.example
```

```

■■■■ docker-compose.yml
■■■■ pom.xml
■■■■ docs/
■ ■■■■ architecture/
■ ■ ■■■■ target-architecture.md
■ ■ ■■■■ architecture-decision-records/
■ ■ ■■■■ data-flow.md
■ ■■■■ clinical/
■ ■ ■■■■ emergency-summary-use-case.md
■ ■ ■■■■ medication-reconciliation-use-case.md
■ ■ ■■■■ clinical-safety-case.md
■ ■■■■ governance/
■ ■ ■■■■ privacy-impact-template.md
■ ■ ■■■■ threat-risk-assessment.md
■ ■ ■■■■ consent-policy.md
■ ■ ■■■■ data-sharing-model.md
■ ■■■■ standards/
■ ■ ■■■■ standards-baseline.md
■ ■ ■■■■ ps-ca-summary-spec.md
■ ■ ■■■■ ca-core-mapping.md
■ ■ ■■■■ cacdi-fhir-mapping.csv
■ ■ ■■■■ terminology-strategy.md
■ ■■■■ runbooks/
■ ■■■■ gateway-outage.md
■ ■■■■ privacy-incident.md
■ ■■■■ emergency-access-review.md
■■■■ implementation-guide/
■ ■■■■ input/
■ ■ ■■■■ profiles/
■ ■ ■■■■ valuesets/
■ ■ ■■■■ examples/
■ ■■■■ README.md
■■■■ test-data/
■ ■■■■ valid/
■ ■ ■■■■ patient-syn-001.json
■ ■ ■■■■ allergy-syn-001.json
■ ■ ■■■■ ps-ca-summary-syn-001.json
■ ■■■■ invalid/
■ ■ ■■■■ invalid-loinc-observation.json
■ ■ ■■■■ incomplete-summary.json
■ ■■■■ identity/
■ ■ ■■■■ match-cases.csv
■ ■■■■ gateway-fixtures/
■ ■■■■ bc-gateway-responses/
■■■■ api/
■ ■■■■ postman/
■ ■■■■ openapi/
■ ■■■■ capability-statement.json
■■■■ infra/
■ ■■■■ docker/
■ ■■■■ terraform/
■ ■■■■ kubernetes/
■ ■■■■ monitoring/
■■■■ scripts/
■ ■■■■ validate-fhir.sh
■ ■■■■ seed-synthetic-data.sh
■ ■■■■ run-contract-tests.sh
■■■■ src/
■ ■■■■ main/
■ ■ ■■■■ java/ca/health/interoperability/
■ ■ ■ ■■■■ InteroperabilityApplication.java
■ ■ ■ ■■■■ config/
■ ■ ■ ■■■■ fhir/
■ ■ ■ ■■■■ provider/
■ ■ ■ ■■■■ operation/
■ ■ ■ ■■■■ validation/
■ ■ ■ ■■■■ mapper/
■ ■ ■ ■■■■ identity/
■ ■ ■ ■■■■ policy/
■ ■ ■ ■■■■ gateway/
■ ■ ■ ■■■■ terminology/
■ ■ ■ ■■■■ provenance/
■ ■ ■ ■■■■ audit/
■ ■ ■ ■■■■ persistence/
■ ■ ■ ■■■■ common/
■ ■ ■■■■ resources/

```

```

■ ■ ■ ■ application.yml
■ ■ ■ ■ db/migration/
■ ■ ■ ■ fhir-packages/
■ ■ ■ ■ test/
■ ■ ■ ■ java/
■ ■ ■ ■ unit/
■ ■ ■ ■ integration/
■ ■ ■ ■ contract/
■ ■ ■ ■ security/
■ ■ ■ ■ conformance/
■ ■ ■ ■ resources/
■ ■ ■ ■ .github/workflows/
■ ■ ■ ■ ci.yml
■ ■ ■ ■ security.yml

```

3. Initial Technology Bill of Materials

Layer	Suggested MVP technology	Responsibility
FHIR service	Java 17, Spring Boot 3.x, HAPI FHIR R4	FHIR REST endpoints, resource providers, validation, Bundle operations
Persistence	PostgreSQL 15+ JSONB, Flyway, JPA	Canonical FHIR JSON plus indexed search projections and history
Authentication	Keycloak, OAuth2/OIDC, SMART on FHIR patterns	Identity, client registration, tokens, claims, roles
Authorization	Spring Security + policy service; later OPA if justified	Purpose, org, role, patient context, consent, break-glass decision
Gateway client	Spring WebClient + Resilience4j	Provincial adapter, timeout, retry, circuit breaker, correlation
Terminology	Approved terminology-service integration + local cache	Validate/translate codes and track terminology releases
Observability	OpenTelemetry + Prometheus/Grafana or Azure Monitor	Metrics, traces, alerts, correlation, source health
DevSecOps	GitHub Actions, Docker, Trivy/Dependabot, SBOM	Build, tests, scans, release gates
Testing	JUnit 5, Mockito, Testcontainers, WireMock, Gatling/JMeter	Unit, integration, gateway contract, performance, resilience
Deployment	Docker locally; secured Canadian cloud container platform for approved pilot	Environment separation, secrets, network, backup, monitoring

4. Example Application Configuration

4.1 application.yml

```
spring:
  application:
    name: canadian-fhir-interoperability
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/fhir_db}
    username: ${DB_USER:fhir_user}
    password: ${DB_PASSWORD:change-me}
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true

server:
  port: 8080

app:
  fhir:
    base-url: ${FHIR_BASE_URL:http://localhost:8080/fhir}
    standards-baseline: ${STANDARDS_BASELINE:2026-mvp}
    gateway:
      timeout-ms: 3000
    cache:
      patient-summary-ttl-seconds: 300

spring.security.oauth2.resourceserver.jwt:
  issuer-uri: ${OIDC_ISSUER_URI:http://localhost:8180/realms/health}
```

4.2 .env.example

```
DB_URL=jdbc:postgresql://postgres:5432/fhir_db
DB_USER=fhir_user
DB_PASSWORD=replace-with-local-secret
OIDC_ISSUER_URI=http://keycloak:8080/realms/health
FHIR_BASE_URL=http://localhost:8080/fhir
BC_GATEWAY_URL=http://wiremock:8081/bc/fhir
# Never commit real keys, PHI, or production values.
```

5. Example FHIR Server Configuration

5.1 HAPI FHIR servlet registration

```
package ca.health.interoperability.config;

import ca.uhn.fhir.rest.server.RestfulServer;
import ca.uhn.fhir.context.FhirContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;

@WebServlet(urlPatterns = "/fhir/*", loadOnStartup = 1)
public class FhirRestServlet extends RestfulServer {
    @Override
    protected void initialize() throws ServletException {
        setFhirContext(FhirContext.forR4());
        registerProvider(getBean(PatientResourceProvider.class));
        registerProvider(getBean(AllergyResourceProvider.class));
        registerProvider(getBean(PatientSummaryOperation.class));
        registerInterceptor(getBean(SmartAuthorizationInterceptor.class));
        registerInterceptor(getBean(AuditInterceptor.class));
    }

    private <T> T getBean(Class<T> type) {
        return SpringContext.getBean(type);
    }
}
```

This example illustrates structure only. In production, configure exception handling, request size limits, CORS policy, response headers, profile validation, and security interceptors deliberately rather than relying on defaults.

5.2 Example Patient resource provider

```
@Component
public class PatientResourceProvider implements IResourceProvider {
    private final PatientService patientService;

    public PatientResourceProvider(PatientService patientService) {
        this.patientService = patientService;
    }

    @Override
    public Class<Patient> getResourceType() {
        return Patient.class;
    }

    @Read
    public Patient read(@IdParam IdType id) {
        return patientService.readAuthorizedPatient(id.getIdPart());
    }

    @Search
    public List<Patient> search(
        @OptionalParam(name = Patient.SP_IDENTIFIER) TokenParam identifier,
        @OptionalParam(name = Patient.SP_FAMILY) StringParam family,
        @OptionalParam(name = Patient.SP_BIRTHDATE) DateParam birthDate) {
        return patientService.searchAuthorized(identifier, family, birthDate);
    }
}
```

The service layer must enforce authorization and identity rules. Do not perform patient lookups solely on name and birth date without policy-defined controls and audit requirements.

6. Example Patient Summary Operation

```
@Component
public class PatientSummaryOperation {
    private final SummaryService summaryService;

    public PatientSummaryOperation(SummaryService summaryService) {
        this.summaryService = summaryService;
    }

    @Operation(name = "$summary", idempotent = true, returnParameters = {
        @OperationParam(name = "return", type = Bundle.class)
    })
    public Bundle summary(@IdParam IdType patientId) {
        AccessContext context = AccessContextHolder.require();
        return summaryService.createAuthorizedSummary(patientId.getIdPart(), context);
    }
}
```

Summary assembly rules

- Validate requester, organization, purpose of use, patient context, consent, and source-jurisdiction rules before retrieval.
- Resolve identity through approved deterministic/candidate-match workflow; do not silently merge low-confidence matches.
- Retrieve local and approved remote resources; capture source, resource version, and freshness.
- Validate resources against the pinned profile and terminology baseline.
- Build a document Bundle using the selected PS-CA structure.
- Represent partial response and source outage as an explicit warning, not an absence assertion.
- Create Provenance and AuditEvent records with correlation ID.

7. Example Security and Policy Structure

7.1 Access context

```
public record AccessContext(
    String practitionerId,
    String organizationId,
    Set<String> roles,
    String purposeOfUse,
    String requestingJurisdiction,
    String correlationId,
    boolean emergencyOverride) {}
```

7.2 Policy decision service

```
@Service
public class AccessPolicyService {
    public PolicyDecision evaluate(AccessContext context,
        String patientId,
        Set<String> resourceTypes,
        String sourceJurisdiction) {
        if (context == null || context.practitionerId() == null) {
            return PolicyDecision.deny("Missing authenticated practitioner");
        }
        if (!"TREATMENT".equals(context.purposeOfUse()) && !context.emergencyOverride()) {
            return PolicyDecision.deny("Purpose of use not permitted");
        }
        // Evaluate organization, role, patient relationship, consent,
        // jurisdiction policy, requested resource types, and emergency obligations.
        return PolicyDecision.permitWithAuditObligation();
    }
}
```

The example deliberately does not encode real PHIPA, PIPA, or jurisdictional consent rules. Those rules must be designed with participating organizations and legal/privacy governance.

7.3 Fine-grained SMART scopes

```
patient/Patient.rs
patient/AllergyIntolerance.rs
patient/Condition.rs
patient/MedicationStatement.rs
patient/MedicationRequest.rs
patient/Observation.rs
patient/DiagnosticReport.rs
patient/Immunization.rs
patient/Patient.$summary
```

Scopes constrain client authorization requests. The server must still perform policy evaluation for each sensitive request.

8. Gateway Adapter Example

```
public interface ProvincialGatewayAdapter {
    GatewayCapabilities discoverCapabilities();
    PatientMatchResult findPatient(PatientIdentityQuery query, AccessContext context);
    RemoteSummaryResult fetchPatientSummary(RemotePatientMatch match,
    AccessContext context);
    GatewayHealth getHealth();
}

@Component
public class BritishColumbiaGatewayAdapter implements ProvincialGatewayAdapter {
    private final WebClient webClient;

    public RemoteSummaryResult fetchPatientSummary(RemotePatientMatch match,
    AccessContext context) {
        // Add correlation ID, client authentication, policy context as approved,
        // timeout, bounded retry, circuit breaker, validation, and source metadata.
        return webClient.get()
            .uri("/Patient/{id}/$summary", match.remotePatientId())
            .retrieve()
            .bodyToMono(Bundle.class)
            .map(bundle -> RemoteSummaryResult.live(bundle, "BC"))
            .block();
    }
}
```

Required gateway response metadata

Field	Purpose
correlationId	Traces the request across the local broker, remote gateway, audit events, and incident handling
sourceJurisdiction	Identifies the authoritative source context
sourceOrganization/system	Identifies the clinical source and technical origin
retrievalTime	Indicates when the exchange occurred
sourceLastUpdated	Indicates the last source update where supplied
dataState	Live, cached, partial, stale, unavailable, invalid, or withheld
validationStatus	Passed, warning, failed, or quarantined
profile/terminology version	Makes interoperability interpretation reproducible

9. Example Persistence Design

9.1 Canonical resource table

```
create table fhir_resource (
  resource_pk uuid primary key,
  resource_type varchar(64) not null,
  logical_id varchar(128) not null,
  version_id varchar(64) not null,
  patient_pk uuid null,
  source_system varchar(256) not null,
  source_resource_id varchar(256) null,
  profile_url varchar(512) null,
  profile_version varchar(128) null,
  last_updated timestamptz not null,
  security_labels jsonb not null default '[]'::jsonb,
  provenance_id varchar(128) null,
  resource_json jsonb not null,
  deleted boolean not null default false,
  unique (resource_type, logical_id, version_id)
);

create index fhir_resource_patient_type_idx
on fhir_resource(patient_pk, resource_type, last_updated desc);
```

9.2 Identity tables

```
create table person_identifier (
  identifier_pk uuid primary key,
  person_pk uuid not null,
  system_uri varchar(512) not null,
  identifier_value varchar(256) not null,
  identifier_type varchar(128),
  assigning_jurisdiction varchar(32),
  status varchar(32) not null,
  verified_at timestamptz,
  source_system varchar(256) not null,
  unique (system_uri, identifier_value)
);

create table identity_match_event (
  event_pk uuid primary key,
  candidate_person_a uuid not null,
  candidate_person_b uuid not null,
  match_method varchar(32) not null,
  match_score numeric(5,4),
  decision varchar(32) not null,
  reviewer_id varchar(128),
  decided_at timestamptz,
  rationale jsonb not null default '{}'::jsonb
);
```

Production schemas require a formal data-model review, retention design, migration plan, database access controls, and alignment with the selected FHIR storage approach. These examples are a starting point only.

10. Example Test Assets

10.1 JUnit service test

```
@ExtendWith(MockitoExtension.class)
class SummaryServiceTest {
    @Mock private AccessPolicyService policy;
    @Mock private PatientRepository patients;
    @InjectMocks private SummaryService service;

    @Test
    void deniesSummaryWhenPolicyDenies() {
        AccessContext ctx = TestContexts.treatmentClinician();
        when(policy.evaluate(eq(ctx), eq("p-1"), anySet(), anyString()))
            .thenReturn(PolicyDecision.deny("Consent restriction"));

        assertThrows(ForbiddenOperationException.class,
            () -> service.createAuthorizedSummary("p-1", ctx));
    }
}
```

10.2 Gateway contract test idea

```
@Test
void returnsPartialOutcomeWhenRemoteGatewayTimesOut() {
    wireMock.stubFor(get(urlPathEqualTo("/bc/fhir/Patient/remote-1/$summary"))
        .willReturn(aResponse().withFixedDelay(5000).withStatus(200)));

    SummaryResponse response = client.retrieveRemoteSummary("remote-1", context);

    assertTrue(response.isPartial());
    assertEquals("BC", response.unavailableSources().getFirst());
    assertNotNull(response.operationOutcome());
}
```

10.3 Synthetic fixture checklist

- Valid Patient plus multiple jurisdiction identifiers.
- Confirmed high-severity allergy with SNOMED CT coding.
- Medication order and patient-reported statement that intentionally conflict.
- LOINC-coded Observation and linked DiagnosticReport.
- PS-CA-like document Bundle with Composition and references.
- Invalid profile fixture, invalid code fixture, unknown code fixture, and missing required data fixture.
- Ambiguous identity fixture: same name/date of birth but different identifiers.
- Consent-denied fixture, break-glass fixture, remote timeout fixture, and stale-cache fixture.

11. Local Development Compose File

Local-only example. Do not treat this as a production configuration. In particular, do not expose databases/Redis publicly or use Keycloak development mode in production.

```
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: fhir_db
      POSTGRES_USER: fhir_user
      POSTGRES_PASSWORD: local-dev-only
    ports: ["5432:5432"]

  keycloak:
    image: quay.io/keycloak/keycloak:23.0
    command: start-dev
```

```

environment:
  KEYCLOAK_ADMIN: admin
  KEYCLOAK_ADMIN_PASSWORD: local-dev-only
  ports: ["8180:8080"]

wiremock:
  image: wiremock/wiremock:3.5.4
  ports: ["8081:8080"]
  volumes:
    - ./test-data/gateway-fixtures:/home/wiremock

fhir-server:
  build: .
  depends_on: [postgres, keycloak, wiremock]
  environment:
    DB_URL: jdbc:postgresql://postgres:5432/fhir_db
    DB_USER: fhir_user
    DB_PASSWORD: local-dev-only
    OIDC_ISSUER_URI: http://keycloak:8080/realms/health
    BC_GATEWAY_URL: http://wiremock:8080/bc/fhir
    ports: ["8080:8080"]

```

12. CI/CD Pipeline Outline

```

name: ci
on: [pull_request, push]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: '17' }
      - run: mvn -B test
      - run: mvn -B verify
      - run: ./scripts/validate-fhir.sh
      - run: ./scripts/run-contract-tests.sh
      - name: Secret scan
      run: <approved-secret-scanner>
      - name: Dependency and image scan
      run: <approved-scanner>
      - name: Generate SBOM
      run: <approved-sbom-tool>
    build:
      needs: verify
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@v4
        - run: docker build -t fhir-server:${{ github.sha }} .
        - run: <sign-and-push-approved-image>

```

Required release gates

- All unit, integration, gateway contract, and security tests pass.
- Pinned FHIR profiles and terminology validations pass.
- No secrets, critical dependency findings, or prohibited PHI test artifacts are detected.
- Database migration has rollback/forward plan.
- Production deployment requires change approval and post-deployment verification.
- Release notes state API, profile, terminology, policy, and compatibility changes.

13. Definition of Done for MVP

Capability	Definition of done
Standards	Exact FHIR R4, PS-CA, CA Core+, CACDI, and terminology baseline is version controlled
Clinical summary	Synthetic patient summary is generated or retrieved, valid against selected profiles, and clinically interpretable
Identity	Multiple identifiers are supported; ambiguous patient candidates do not automatically merge
Security	SMART/OIDC authentication plus policy decision and audit are enforced for protected data
Provenance	Every returned high-priority item identifies source and freshness
Federation	A simulated BC adapter supports discovery, patient match, summary retrieval, timeout, and partial response
Safety	Stale, invalid, conflicting, unavailable, and withheld data are visibly distinguished
Testing	Automated unit, integration, profile, terminology, identity, policy, and gateway tests are part of CI
Operations	Local observability, correlation IDs, error handling, and incident runbook draft exist
Demo	Authorized synthetic Ontario clinician retrieves a synthetic BC summary with audit evidence

14. Official Standards Links

Item	Link
HL7 FHIR R4	https://hl7.org/fhir/R4/index.html
CA Core+	https://simplifier.net/guide/ca-core
PS-CA overview	Infoway PS-CA FHIR Implementation Guide
PS-CA publication area	https://simplifier.net/guide/ps-ca
CACDI overview	CIHI CACDI overview
CACDI Version 2	CACDI v2 PDF
Canadian FHIR Registry	https://simplifier.net/organization/canadianfhirregistry
Infoway FHIR community	InfoCentral FHIR Implementations

End of engineering appendix.